

ECE-9413 Final Report Guidelines

Assignment 1: Negacyclic NTT + Assignment 2: SumCheck Prover

1. Format and Submission

Use the same two-column IEEE-style report format as your weekly paper submissions. Submit one PDF covering both assignments. The target length is **7–8 pages** and no more than 11 pages, excluding references.

If you worked in a group, submit one shared report, list every group member on the first page, and have each member upload the same PDF.

Course TAs will run the submitted code. Performance claims must be traceable to the submitted implementation, the reported commands, the benchmark settings, and the hardware used. If you used hardware beyond the standard course GPU setup, state that clearly with the corresponding results.

Important SumCheck Note: Please do not use the round challenges ahead of time to precompute the per-round tables. In other words, MLE Updates to reduce the table sizes using the provided random challenges must happen after/between rounds. As noted in the `sumcheck_intro.md`, we provided the challenges ahead of time and omitted hashing in between rounds. Hashing is left as an optional bonus track. If you precompute the tables with MLE Update, we will deduct points.

2. Abstract

The abstract should give a compact account of the full project: the two implemented kernels, required tests passed, headline CPU-to-GPU speedups, representative GPU latency or throughput numbers, optimizations that changed the measurements, ideas that did not work, bonus tracks completed, and the main remaining limitation.

3. Introduction

Include the motivation for accelerating the two kernels, enough algorithmic context to make the implementation choices understandable, and the main contributions of the work.

Some questions you might want to answer:

- Why are NTT and SumCheck relevant to the target workloads?
- What makes these kernels different from ordinary dense floating-point GPU workloads?
- Which tests passed, and which benchmark numbers summarize the implementation?
- Which implementation idea mattered most for each assignment?

These are just suggestions, so feel free to use or not use them.

4. Assignment 1: Negacyclic NTT

4.1. Design and Optimization

Briefly introduce the NTT algorithm, enough to properly motivate your implementation and optimization. You can use the following questions to help structure this section. Once again, these are just suggestions; you do not need to answer all of them. Moreover, avoid dedicating one paragraph per question below and leaving it at that. Think back to how the papers you read structured their results. What did you like or not like?

- Which transform structure did you implement: direct evaluation, Cooley–Tukey, Stockham, twisted cyclic NTT, or another schedule?
- Why did that structure fit the workload: memory coalescing, fewer permutations, simpler batching, shorter compilation, easier validation, or better measured latency?
- Do you believe the NTT was compute limited or bandwidth limited? Did this vary with batch size and degree? Could this be visualized with your own roofline plot?
- How were `mod_add`, `mod_sub`, and `mod_mul` implemented on the hot path?
- Which multiplication and reduction method was used: plain modulo, widening, Barrett reduction, Montgomery reduction, signed Montgomery-style arithmetic, or another method?
- Which JAX structure controlled the lowering: `jit`, `vmap`, `lax.scan`, `lax.fori_loop`, reshaping, transposition, Pallas, or another approach?
- What optimizations were added between the first correct version and the final measured version?
- Which attempted optimizations helped only small sizes, increased compile time, used too much memory, or complicated correctness?

4.2. Correctness

Report the commands run and their pass-or-fail status. Include the relevant Assignment 1 checks:

```
uv run pytest
uv run pytest --logn 10 --batch 4
uv run python -m tests.benchmark --tests --logn 10 --batch 4
```

A small table is enough. For any failure, identify the failing configuration: `logn`, batch size, backend, dtype path, or table layout.

4.3. Performance

Include at least one NTT performance figure. Suitable figures include a heatmap over `logn` and batch size, a sweep over `logn` at fixed batch size, or a sweep over batch size at fixed `logn`. Color heatmaps by a single defined metric such as median latency or throughput. You may include more figures than this if you have meaningful optimizations that you'd like to show. Use this section to discuss and explore your results.

5. Assignment 2: SumCheck

5.1. Design and Optimization

Use the following questions to select the technical details that best explain the implementation and its scaling behavior. Once again, these are just suggestions.

- Which modular arithmetic ideas transferred from Assignment 1, and which did not?
- Did you implement Barrett/Montgomery/other modular arithmetic primitives? and how?
- Did you implement special functions for the four base expressions, or implement a general function to support any expression?
- How did expression degree change the number of round-evaluation points?
- How did performance change...
 - as the polynomial degree increased?
 - as the number of terms increased?
 - between composite polynomials that did and did not have repeated constituent polynomials?
 - as the number of variables (which controls the size of the tables) increased from 4, to 16, to 20?
- How did you implement MLE Update? Did you use a full modular multiplication, or did you use a cheaper path?
- Did you perform MLE Updates strictly after a SumCheck round, or did you include it during a round (similar to how NoCap conditionally folded it into their computation path in Listing 1)? Did you notice any improvement/degradation to performance?
- Which choices worked for `vars4` but became slow, memory-heavy, or compile-heavy for `vars16` and `vars20`?
- Did you attempt any bonus tracks? If so, please just list them here.
- If you attempted bonus tracks for 64-bit and/or 128-bit, what did you have to account for in order to support computations that exceeded the supported 64-bit datatypes? Feel free to provide diagrams as part of your analysis if it helps.
- If you attempted any bonus tracks with the advanced polynomials or polynomials from research papers, did you notice any trends in performance? Feel free to provide plots/tables as part of your analysis.
- If you attempted any other bonus tracks not mentioned, provide details here.

5.2. READMEs

This section helps us understand if the resources were helpful. If you did not read a section or did not find a resource useful, please indicate so. You will **not** be penalized for any answers in this section, but this section is compulsory.

- Did you find the main README helpful? Provide suggestions if you want to.

- Did you read the SumCheck README? If so, did it help to understand the algorithm, or make things a bit confusing?
- Did you read any of other reference materials (either provided, or some you found on your own)? If so, were they helpful, or confusing?
- Did you try to implement custom cases? If so, was the README helpful or confusing to understand?

5.3. Correctness

Report the required 32-bit commands and their pass-or-fail status:

```
uv run pytest --bits 32 --num-vars 4
uv run pytest --bits 32 --num-vars 16
uv run pytest --bits 32 --num-vars 20
```

5.4. Performance

5.4.1 Figures

Include at least two SumCheck performance figures. Some suggestions:

- a line plot of runtime performance over a fixed problem size for different polynomial expressions (one line per expression)
- a line plot of the runtime performance for a fixed expression over different problem sizes (one line per problem size)
- a 2D (or 3D) heatmap of performance across different problem sizes/expressions
- A table comparing runtime numbers between CPU/GPU (and TPU if you were able to run on a TPU)
- A bar chart of speedups achieved over CPU when running on GPU
- An ablation bar chart of speedups achieved as you tried different optimizations, showing which optimizations gave you what kinds of speedups
- A pie chart of time to complete SumCheck rounds versus MLE updates
- A pie chart with the breakdown of compile time versus runtime

5.4.2 Benchmarks

Use the required benchmark commands as the main baseline:

```
uv run python -m tests.benchmark --bench --bits 32 --num-vars 4 --runs 8 --warmup 3
uv run python -m tests.benchmark --bench --bits 32 --num-vars 16 --runs 8 --warmup 3
uv run python -m tests.benchmark --bench --bits 32 --num-vars 20 --runs 8 --warmup 3
```

Sort the expressions by their latency and explain why you think they behave that way. Is it related to polynomial degree, number of terms, table size, compilation behavior, or another effect? Share which optimization(s) (if attempted) that yielded the best speedups. For bonus tracks, please include commands, pass status, hardware, and measured numbers.

6. Cross-Assignment Discussion

Compare the two kernels directly and connect to broader lessons from the class.

Which modular arithmetic ideas transferred from NTT to SumCheck? Which project was more sensitive to multiplication speed? Which project was more sensitive to memory movement? Did the same JAX patterns help both projects, or did the kernels require different structures?

7. Optional AI Use Reflection

This section is optional, but counts towards the total page count. If included, focus on how you effectively used AI to improve the code's performance in either of your assignments. Or, if you avoided its use, why?

Possible topics:

- Where AI helped most: background learning, first implementation, debugging, JAX errors, profiling, benchmark design.
- One suggestion that changed the code or experiment.
- One plausible suggestion that was wrong, slow, un-JAX-like, or incompatible with the API.
- How the prompts improved over time.
- One piece of advice for next year's students using AI on this project.

Be specific here!

8. Conclusion

End with concrete results: required tests passed, the headline NTT number, the headline SumCheck number, the most important optimization in each implementation, and the main limitation that remains.

9. Grading

Component	Weight	What we look for
Assignment 1: NTT	30%	Correct implementation, preserved API, passing tests, benchmark evidence, and real optimization effort.
Assignment 2: SumCheck	50%	Correct 32-bit required track, benchmark evidence, real optimization effort, README feedback, and clearly demonstrated bonus work where present.
Report quality	20%	Clear organization, self-contained figures, exact commands, honest limitations, and design choices tied to measured results. The AI-use reflection is optional.